Esercizio di progettazione Laboratorio di AI

Aula Virtuale 28-10-2026

Si vuole realizzare un assistente intelligente per la gestione automatica di **recensioni e richieste clienti in ambito e-commerce**.

Il sistema deve essere capace di:

1. analizzare un testo breve scritto da un cliente;
2. classificarlo in una categoria semantica;
3. generare una risposta automatica coerente;
4. simulare un comportamento agentico, cioè eseguire un'azione diversa in funzione del contenuto del testo.

Dal punto di vista del corso, il problema integra tre blocchi fondamentali:

- **IA discriminativa**: classificazione supervisionata del testo;
- **IA generativa**: generazione di risposta mediante Transformer pre-addestrato;
- **IA agentica**: scelta ed esecuzione di un'azione automatica in base alla classe predetta.

Questa impostazione è pienamente coerente con il programma del laboratorio, che include apprendimento supervisionato, NLP, IA generativa e IA agentica.

1. **Architettura logica del sistema**

Il prototipo può essere descritto come una pipeline composta dai seguenti moduli:

- **input testuale**: recensione o messaggio cliente;
- **preprocessing leggero**;
- **classificatore supervisionato**;
- **etichetta di classe**:
    - positiva
    - negativa
    - richiesta_supporto
- **modulo generativo Transformer**;
- **risposta automatica** oppure **sintesi**;
- **agente software**;
- **azione simulata**:
    - registra feedback positivo,
    - inoltra al reparto qualità,
    - apre ticket di assistenza.

Questa scomposizione è utile perché separa chiaramente:

- il problema della **comprensione del testo**;
- il problema della **generazione linguistica**;
- il problema della **decisione operativa**.

2. **Dataset di esempio**

Poiché l'esercizio può essere svolto con dati simulati, costruiamo un piccolo dataset etichettato manualmente.

    a. **Dataset**

```
dataset = [
  # positiva
  ("Prodotto eccellente, consegna veloce e qualità ottima.", "positiva"),
  ("Sono molto soddisfatto dell'acquisto, lo consiglio vivamente.", "positiva"),
```

```
    ("Servizio impeccabile e articolo conforme alla descrizione.", "positiva"),
    ("Esperienza perfetta, acquisterò di nuovo da questo negozio.", "positiva"),

    # negativa
    ("Il prodotto è arrivato danneggiato, pessima esperienza.", "negativa"),
    ("Non funziona dopo pochi giorni, sono molto deluso.", "negativa"),
    ("Qualità scarsa rispetto al prezzo pagato.", "negativa"),
    ("Consegna in ritardo e assistenza clienti poco disponibile.", "negativa"),

    # richiesta_supporto
    ("Non riesco a configurare il dispositivo, potete aiutarmi?", "richiesta_supporto"),
    ("Ho bisogno di assistenza per effettuare il reso.", "richiesta_supporto"),
    ("Non capisco come attivare il prodotto acquistato.", "richiesta_supporto"),
    ("Vorrei supporto per sostituire l'articolo ricevuto.", "richiesta_supporto"),
]
```

b. Osservazione metodologica

Questo dataset è:

- molto piccolo;
- utile soltanto come **prototipo dimostrativo**;
- non sufficiente per una generalizzazione robusta in un contesto reale.

In un sistema industriale servirebbero:

- più dati;
- maggiore varietà linguistica;
- una validazione più rigorosa;
- gestione di testi ambigui o misti.

3. Classificazione del testo

L'esercizio richiede un modello supervisionato. Una scelta semplice, corretta e adatta a un laboratorio universitario è:

- **TF-IDF + Logistic Regression**

Questa soluzione è appropriata perché:

1. è facile da implementare;
2. è interpretabile;
3. funziona bene su testi brevi;
4. rispetta pienamente la richiesta di classificazione supervisionata.

    a. **Addestramento del classificatore**

```
from sklearn.model_selection import train_test_split
from sklearn.pipeline import Pipeline
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import classification_report

texts = [x[0] for x in dataset]
labels = [x[1] for x in dataset]

X_train, X_test, y_train, y_test = train_test_split(
    texts,
    labels,
    test_size=0.25,
    random_state=42,
    stratify=labels
)

clf = Pipeline([
    ("tfidf", TfidfVectorizer(lowercase=True, ngram_range=(1, 2))),
    ("model", LogisticRegression(max_iter=1000))
])
```

```
clf.fit(X_train, y_train)


pred = clf.predict(X_test)
print(classification_report(y_test, pred))
```

b. Commento metodologico

La pipeline usa:

- **TF-IDF** per trasformare i testi in vettori numerici;
- **Logistic Regression** per la classificazione multiclasse.

Questa scelta è molto adatta a recensioni e richieste brevi, perché parole ed espressioni chiave sono spesso sufficienti a distinguere le classi:

- “eccellente”, “soddisfatto”, “perfetta” → positiva
- “danneggiato”, “deluso”, “scarsa” → negativa
- “aiutarmi”, “assistenza”, “reso” → richiesta_supporto

4. Funzione di classificazione

Definiamo ora una funzione che, dato un testo, restituisca:

- la classe predetta;
- le probabilità associate a ciascuna classe.

```
def classify_document(text, classifier):
  label = classifier.predict([text])[0]
  probs = classifier.predict_proba([text])[0]
  classes = classifier.classes_
  confidence = {classes[i]: float(probs[i]) for i in range(len(classes))}
  return label, confidence
```

a. Esempio d’uso

```
doc = "Il pacco è arrivato in ritardo e il prodotto è difettoso."
label, confidence = classify_document(doc, clf)

print("Classe predetta:", label)
```

```
print("Confidenze:", confidence)
```

b. Output atteso

Un possibile output è:

```
Classe predetta: negativa
Confidenze: {
  'negativa': 0.74,
  'positiva': 0.08,
  'richiesta_supporto': 0.18
}
```

5. Generazione di testo con Transformer

La seconda parte richiede l'uso di un **modello Transformer pre-addestrato** per generare testo.

Possiamo usare, ad esempio, una pipeline di Hugging Face con un modello text2text-generation, come google/flan-t5-base.

a. Inizializzazione del generatore

```
from transformers import pipeline

generator = pipeline(
  "text2text-generation",
  model="google/flan-t5-base"
)
```

6. Generazione di una sintesi

Una prima possibilità consiste nel generare una sintesi del testo ricevuto.

```
def summarize_document(text):
  prompt = f"Riassumi in modo chiaro e conciso la seguente recensione del cliente:\n\n{text}"
  result = generator(prompt, max_new_tokens=60, do_sample=False)
  return result[0]["generated_text"]
```

a. Esempio

doc = "Il prodotto è arrivato con due giorni di ritardo e inoltre presenta un difetto di fabbricazione."

print(summarize_document(doc))

b. Possibile output

Il cliente segnala un ritardo nella consegna e un difetto del prodotto ricevuto.

7. Generazione di una risposta automatica

La seconda possibilità, più vicina a un sistema reale, consiste nel generare una **risposta automatica coerente con la categoria del testo**.

a. Funzione di generazione

```
def generate_response(text, label, detailed=False):
  if detailed:
    prompt = f"""
Sei un assistente clienti di una piattaforma e-commerce.

Categoria del messaggio: {label}

Messaggio del cliente:
{text}

Scrivi una risposta professionale, cortese e coerente con la categoria.

Regole:
- Se la categoria è positiva, ringrazia il cliente per il feedback e valorizza la soddisfazione.
- Se la categoria è negativa, esprimi dispiacere, mostra empatia e informa che il caso sarà inoltrato al reparto qualità.
- Se la categoria è richiesta_supporto, comunica che la richiesta è stata presa in carico e che verrà aperto un ticket di assistenza.
- Mantieni un tono formale, chiaro e sintetico.
```

"""

else:

prompt = f"Scrivi una risposta al seguente messaggio del cliente, categoria {label}: {text}"

result = generator(prompt, max_new_tokens=80, do_sample=False)

return result[0]["generated_text"]

8. **Effetto della variazione del prompt**

Questa parte è importante perché consente di mostrare che la qualità dell'output dipende dalla formulazione del prompt.

Supponiamo di avere il seguente documento:

doc = "Il prodotto è arrivato danneggiato e sono molto deluso."

label, _ = classify_document(doc, clf)

a. **Prompt generico**

print(generate_response(doc, label, detailed=False))

Possibile output

Ci dispiace per il problema riscontrato. La sua segnalazione sarà presa in carico.

b. **Prompt dettagliato**

print(generate_response(doc, label, detailed=True))

Possibile output

Gentile cliente, ci dispiace sinceramente per il problema riscontrato con il prodotto ricevuto. Comprendiamo la sua insoddisfazione e abbiamo preso in carico la sua segnalazione. Il caso sarà inoltrato al reparto qualità per le verifiche necessarie e la ricontatteremo al più presto.

9. **Osservazioni**

La differenza è chiara:

- il **prompt generico** lascia maggiore libertà al modello;
- il **prompt dettagliato** controlla meglio:
    - tono;
    - contenuto;

- struttura;
- comportamento atteso.

Quindi la **prompt engineering** è un elemento centrale nel controllo della generazione testuale.

10. Simulazione del comportamento agentico

Il sistema non deve solo comprendere e generare testo, ma anche **agire**.

La politica decisionale dell'agente sarà:

- positiva → registra feedback positivo;
- negativa → inoltra al reparto qualità;
- richiesta_supporto → apre ticket di assistenza.

Questa è una forma elementare ma corretta di IA agentica, perché include:

1. percezione dell'input;
2. interpretazione;
3. decisione;
4. esecuzione di un'azione.

a. Funzioni Python per le azioni

```
def register_positive_feedback(text):
  return {
    "azione": "registrazione_feedback_positivo",
    "stato": "eseguito",
    "dettaglio": f"Feedback positivo registrato: {text}"
  }


def forward_to_quality_department(text):
  return {
    "azione": "inoltro_reparto_qualita",
    "stato": "eseguito",
    "dettaglio": f"Segnalazione inoltrata al reparto qualità: {text}"
```

```
    }


def open_support_ticket(text):
    return {
        "azione": "apertura_ticket_assistenza",
        "stato": "eseguito",
        "dettaglio": f"Ticket di assistenza aperto per: {text}"
    }
```

b. Logica decisionale dell'agente

```
def agent_action(text, label):
    if label == "positiva":
        return register_positive_feedback(text)
    elif label == "negativa":
        return forward_to_quality_department(text)
    elif label == "richiesta_supporto":
        return open_support_ticket(text)
    else:
        return {
            "azione": "nessuna",
            "stato": "errore",
            "dettaglio": "Categoria non riconosciuta"
        }
```

c. Commento

Questa soluzione è **rule-based**, cioè basata su regole esplicite.

È una scelta molto appropriata in un prototipo, perché consente di separare:

- la parte statistica: classificazione del testo;
- la parte decisionale: regole dell'agente.

In una versione più avanzata si potrebbero introdurre:

- soglie di confidenza;

- gestione dell’incertezza;
- supervisione umana;
- integrazione con API reali di CRM, ticketing o customer care.

11. Pipeline completa end-to-end

Uniamo ora tutti i moduli in un’unica funzione.

```
def intelligent_ecommerce_assistant(document, classifier, detailed_prompt=True):
    # 1. classificazione
    label, confidence = classify_document(document, classifier)

    # 2. generazione risposta
    response = generate_response(document, label, detailed=detailed_prompt)

    # 3. azione agentica
    action_result = agent_action(document, label)

    return {
        "documento": document,
        "categoria_predetta": label,
        "confidenza": confidence,
        "risposta_generata": response,
        "azione_agente": action_result
    }
```

12. Esempio completo di esecuzione

```
doc1 = "Non riesco ad attivare il dispositivo acquistato, potete aiutarmi?"
result = intelligent_ecommerce_assistant(doc1, clf, detailed_prompt=True)

for k, v in result.items():
```

```
print(f"{k}: {v}")
```

a. **Output atteso**

documento: Non riesco ad attivare il dispositivo acquistato, potete aiutarmi?

categoria_predetta: richiesta_supporto

confidenza: {'negativa': 0.11, 'positiva': 0.06, 'richiesta_supporto': 0.83}

risposta_generata: Gentile cliente, la sua richiesta di assistenza è stata presa in carico. Abbiamo aperto un ticket di supporto per aiutarla nell'attivazione del dispositivo e la contatteremo al più presto.

azione_agente: {'azione': 'apertura_ticket_assistenza', 'stato': 'eseguito', 'dettaglio': 'Ticket di assistenza aperto per: Non riesco ad attivare il dispositivo acquistato, potete aiutarmi?'}

13. Versione migliorata con soglia di confidenza

Per rendere il prototipo più realistico, possiamo introdurre una **soglia di confidenza**.

Se la probabilità massima è troppo bassa, l'agente non esegue azioni automatiche ma richiede revisione umana.

a. **Funzione aggiornata**

```
def intelligent_ecommerce_assistant_with_threshold(document, classifier, threshold=0.60, detailed_prompt=True):
    label, confidence = classify_document(document, classifier)
    max_conf = max(confidence.values())

    if max_conf < threshold:
        return {
            "documento": document,
            "categoria_predetta": label,
            "confidenza": confidence,
            "risposta_generata": "Il messaggio richiede una revisione manuale da parte di un operatore.",
            "azione_agente": {
                "azione": "revisione_umana",
                "stato": "in_attesa",
```

```
            "dettaglio": "Confidenza troppo bassa per azione automatica"
        }
    }

    response = generate_response(document, label, detailed=detailed_prompt)
    action_result = agent_action(document, label)

    return {
        "documento": document,
        "categoria_predetta": label,
        "confidenza": confidence,
        "risposta_generata": response,
        "azione_agente": action_result
    }
```

b. Vantaggio

Questo accorgimento migliora l'affidabilità del sistema, perché evita azioni automatiche su casi incerti o ambigui.

14. Discussione critica della soluzione

a. Punti di forza

La soluzione proposta ha diversi punti di forza:

- architettura modulare e chiara;
- classificazione supervisionata coerente con la traccia;
- uso di Transformer per la generazione linguistica;
- simulazione esplicita del comportamento agentico;
- possibilità di mostrare l'effetto della variazione del prompt;
- facile estendibilità.

b. Limiti

I principali limiti sono:

- dataset molto piccolo;
- assenza di validazione sperimentale robusta;
- possibili errori di classificazione su testi ambigui;
- generazione non sempre perfettamente controllabile;
- agente basato su regole semplici, non su pianificazione complessa.

15. Miglioramenti futuri

In un sistema reale si potrebbero introdurre:

- dataset più ampio e bilanciato;
- cross-validation;
- fine-tuning di un Transformer per la classificazione;
- RAG con base di conoscenza aziendale;
- integrazione con sistemi reali di help desk e CRM;
- logging e monitoraggio delle decisioni dell'agente.

16. Conclusione

Il prototipo soddisfa i tre requisiti fondamentali dell'esercizio:

- **classificazione supervisionata del testo**;
- **generazione di output linguistico con Transformer pre-addestrato**;
- **simulazione di comportamento agentico**.

La soluzione è coerente con gli obiettivi del corso, perché integra in un'unica pipeline software:

- metodi di IA discriminativa;
- NLP generativo;
- logica decisionale agentica.

Come prova di laboratorio è una soluzione corretta, modulare, comprensibile ed estendibile, nello stesso spirito dell'esempio allegato.

17. Codice completo

```
from sklearn.model_selection import train_test_split
from sklearn.pipeline import Pipeline
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import classification_report
from transformers import pipeline

# ========================
# 1. Dataset simulato
# ========================
dataset = [
    ("Prodotto eccellente, consegna veloce e qualità ottima.", "positiva"),
    ("Sono molto soddisfatto dell'acquisto, lo consiglio vivamente.", "positiva"),
    ("Servizio impeccabile e articolo conforme alla descrizione.", "positiva"),
    ("Esperienza perfetta, acquisterò di nuovo da questo negozio.", "positiva"),

    ("Il prodotto è arrivato danneggiato, pessima esperienza.", "negativa"),
    ("Non funziona dopo pochi giorni, sono molto deluso.", "negativa"),
    ("Qualità scarsa rispetto al prezzo pagato.", "negativa"),
    ("Consegna in ritardo e assistenza clienti poco disponibile.", "negativa"),

    ("Non riesco a configurare il dispositivo, potete aiutarmi?", "richiesta_supporto"),
    ("Ho bisogno di assistenza per effettuare il reso.", "richiesta_supporto"),
    ("Non capisco come attivare il prodotto acquistato.", "richiesta_supporto"),
    ("Vorrei supporto per sostituire l'articolo ricevuto.", "richiesta_supporto"),
]
```

```
texts = [x[0] for x in dataset]
labels = [x[1] for x in dataset]

# =========================
# 2. Classificatore supervisionato
# =========================
X_train, X_test, y_train, y_test = train_test_split(
    texts,
    labels,
    test_size=0.25,
    random_state=42,
    stratify=labels
)

clf = Pipeline([
    ("tfidf", TfidfVectorizer(lowercase=True, ngram_range=(1, 2))),
    ("model", LogisticRegression(max_iter=1000))
])

clf.fit(X_train, y_train)

pred = clf.predict(X_test)
print("=== REPORT DI CLASSIFICAZIONE ===")
print(classification_report(y_test, pred))

def classify_document(text, classifier):
    label = classifier.predict([text])[0]
    probs = classifier.predict_proba([text])[0]
    classes = classifier.classes_
```

```
    confidence = {classes[i]: float(probs[i]) for i in range(len(classes))}
    return label, confidence

# ========================
# 3. Modello generativo Transformer
# ========================
generator = pipeline(
    "text2text-generation",
    model="google/flan-t5-base"
)

def summarize_document(text):
    prompt = f"Riassumi in modo chiaro e conciso la seguente recensione del cliente:\n\n{text}"
    result = generator(prompt, max_new_tokens=60, do_sample=False)
    return result[0]["generated_text"]

def generate_response(text, label, detailed=False):
    if detailed:
        prompt = f"""
Sei un assistente clienti di una piattaforma e-commerce.

Categoria del messaggio: {label}

Messaggio del cliente:
{text}

Scrivi una risposta professionale, cortese e coerente con la categoria.

Regole:
```

- Se la categoria è positiva, ringrazia il cliente per il feedback e valorizza la soddisfazione.

- Se la categoria è negativa, esprimi dispiacere, mostra empatia e informa che il caso sarà inoltrato al reparto qualità.

- Se la categoria è richiesta_supporto, comunica che la richiesta è stata presa in carico e che verrà aperto un ticket di assistenza.

- Mantieni un tono formale, chiaro e sintetico.

"""

```
    else:
        prompt = f"Scrivi una risposta al seguente messaggio del cliente, categoria {label}: {text}"

    result = generator(prompt, max_new_tokens=80, do_sample=False)
    return result[0]["generated_text"]

# ==========================
# 4. Azioni agentiche
# ==========================
def register_positive_feedback(text):
    return {
        "azione": "registrazione_feedback_positivo",
        "stato": "eseguito",
        "dettaglio": f"Feedback positivo registrato: {text}"
    }

def forward_to_quality_department(text):
    return {
        "azione": "inoltro_reparto_qualita",
        "stato": "eseguito",
        "dettaglio": f"Segnalazione inoltrata al reparto qualità: {text}"
    }
```

```
def open_support_ticket(text):
    return {
        "azione": "apertura_ticket_assistenza",
        "stato": "eseguito",
        "dettaglio": f"Ticket di assistenza aperto per: {text}"
    }

def agent_action(text, label):
    if label == "positiva":
        return register_positive_feedback(text)
    elif label == "negativa":
        return forward_to_quality_department(text)
    elif label == "richiesta_supporto":
        return open_support_ticket(text)
    else:
        return {
            "azione": "nessuna",
            "stato": "errore",
            "dettaglio": "Categoria non riconosciuta"
        }

# ========================
# 5. Pipeline completa
# ========================
def intelligent_ecommerce_assistant(document, classifier, detailed_prompt=True):
    label, confidence = classify_document(document, classifier)
    response = generate_response(document, label, detailed=detailed_prompt)
    action_result = agent_action(document, label)
```

```
    return {
        "documento": document,
        "categoria_predetta": label,
        "confidenza": confidence,
        "risposta_generata": response,
        "azione_agente": action_result
    }


# ========================
# 6. Versione con soglia
# ========================
def intelligent_ecommerce_assistant_with_threshold(document, classifier, threshold=0.60, detailed_prompt=True):
    label, confidence = classify_document(document, classifier)
    max_conf = max(confidence.values())


    if max_conf < threshold:
        return {
            "documento": document,
            "categoria_predetta": label,
            "confidenza": confidence,
            "risposta_generata": "Il messaggio richiede una revisione manuale da parte di un operatore.",
            "azione_agente": {
                "azione": "revisione_umana",
                "stato": "in_attesa",
                "dettaglio": "Confidenza troppo bassa per azione automatica"
            }
```

```
    }

  response = generate_response(document, label, detailed=detailed_prompt)
  action_result = agent_action(document, label)

  return {
    "documento": document,
    "categoria_predetta": label,
    "confidenza": confidence,
    "risposta_generata": response,
    "azione_agente": action_result
  }

# ========================
# 7. Esempio d'uso
# ========================
doc = "Non riesco ad attivare il dispositivo acquistato, potete aiutarmi?"
result = intelligent_ecommerce_assistant(doc, clf, detailed_prompt=True)

print("\n=== RISULTATO FINALE ===")
for k, v in result.items():
  print(f"{k}: {v}")

print("\n=== DIFFERENZA TRA PROMPT ===")
print("Prompt generico:")
print(generate_response(doc, result["categoria_predetta"], detailed=False))

print("\nPrompt dettagliato:")
print(generate_response(doc, result["categoria_predetta"], detailed=True))
```